

cloudborch / develop — Step 5 outline

Configurable *explicit* regularisation on the step-4 PINN: autograd space+time smoothness to tame the background degeneracy, and per-parameter soft-hinge bounds on every quantity. Deliverable of `instructions.md` step 5. (Step 6 = deployment.)

1. OBJECTIVE

The step-4 fit reaches the noise floor but the background/cloud split is degenerate (background grows emission spikes the clouds cancel). Add explicit penalties to the loss — the PINN-native way, since parameters are $p_k = f_\theta(x, y, t)$: penalise the *true* coordinate derivatives (autograd). This gives per-parameter *control* (smooth the background hard, the clouds lightly) on top of the implicit Fourier smoothness. Loss becomes

$$\mathcal{L} = \chi^2 + \mathcal{R}_{\text{smooth}} + \mathcal{R}_{\text{bound}}.$$

2. SMOOTHNESS (AUTOGRAD COORDINATE-DERIVATIVES)

For each field p_k , with $\mathbf{v} = (x, y, t)$ carrying `requires_grad`,

$$\mathcal{R}_{\text{smooth}} = \sum_k \left[w_k^{xy} \langle (\partial_x p_k)^2 + (\partial_y p_k)^2 \rangle + w_k^t \langle (\partial_t p_k)^2 \rangle \right],$$

each $\partial p_k / \partial \mathbf{v}$ from `torch.autograd.grad(..., create_graph=True)`. Mesh-free and exact, so it works with *random minibatches* (needed for the step-6 full-cube run). Cost: one extra backward through the *MLP* only (not the physics), so mild — the Voigt forward still dominates.

3. BOUNDS (SOFT HINGES, PER PARAMETER)

Reusing the `utils.regu_min/regu_max` idea, per quantity with optional (ℓ_k, u_k) ,

$$\mathcal{R}_{\text{bound}} = \sum_k \left[w_k^\ell \langle \text{ReLU}(\ell_k - p_k)^2 \rangle + w_k^u \langle \text{ReLU}(p_k - u_k)^2 \rangle \right].$$

Applies to *all* cloud params and the background. The background **no-emission** bound is an upper hinge on the reconstructed *spectrum*, $\langle \text{ReLU}(b(x, y, t, \lambda) - 1)^2 \rangle_\lambda$. Since bounds are now hinges, the step-4 *strict* transforms relax to positivity-only (softplus on $S, \Delta\tau, \Delta v$ — keeps the sign-degeneracy fix); v_{los} and the background coeffs go free, their limits owned by the hinges.

4. THE POINT: CONTROL THE BACKGROUND

Default config: **strong** w^{xy}, w^t on the 6 background coefficients + the no-emission bound — the localized emission spikes are high-frequency, so heavy space+time smoothing plus the cap should suppress them; **light** smoothing + physical hinges ($S, \Delta\tau \geq 0$; $\Delta v \geq 8$; $|v_{\text{los}}| \leq v_{\text{max}}$) on the clouds. Backup lever if needed: an anchor $\|\mathbf{c} - \mathbf{c}_{\text{ref}}\|^2$ toward the *filament-free* last-10-frame background (*not* the observed projection — that already contains the absorption; the freeze test showed it fails).

5. INTERFACE & TUNING

A `regularizers.py` (`smoothness_penalty`, `hinge_bounds`, `no_emission`) + a REG config dict per group $\{w^{xy}, w^t, \ell, u, w^\ell, w^u\}$; reuses `composite_synth/chi2_per_pixel`. The real work is *weight balancing* (data vs each term) — scan a few, watch that the background loses its emission spikes while χ^2 stays near noise and v_{los} maps gain structure.

Plan only — no code until you approve. Then `regularizers.py` + notebook update + a weight scan. **Step 6** = deployment prep: notebook→script, config/CLI, `device=cuda`, coordinate minibatching, checkpointing/resume.